Complete Contracts for EiffelBase

Marco Zietzling

02-906-998
Semester Thesis
Summer 2007

Chair of Software Engineering
Department of Computer Science
ETH Zürich

Bernd Schoeller
Prof. Bertrand Meyer

Abstract

Acknowledgments

I would like to thank my supervisor Bernd Schoeller for his competent support and helpful advices. He always had time for answering my questions and giving hints concerning implementation and general library design. I also would like to thank the other members of the ballet team (Raphael Mack and Jason (Yi) Wei).

Additionally I would like to thank my girlfriend, my family and my friends for the support during my studies and this thesis.

Contents

Abstract **iii**

Acknowledgments **v**

1 Introduction **1**
1.1 Motivation 1
1.2 Related Work 2
1.3 Overview 2

2 Frames **3**
2.1 Frame problem 3
2.2 Dynamic Frames 4
2.3 Dynamic Frames in Eiffel 6

3 Models **7**
3.1 Mathematical Model Library (MML) 8
3.2 Models in Eiffel 8

4 New library with Complete contracts **13**
4.1 EiffelBase with Complete Contracts 13
4.1.1 Differences to EiffelBase 13
4.2 Linked List class 15

5 Results and Conclusions **17**
5.1 Usage of Frames and Models in Eiffel 17

Bibliography **18**

Chapter 1

Introduction

1.1 Motivation

The contracts currently used in Eiffel (cf. [Mey92b, Mey92a, Mey97]) are strong under-specifications. There are always some properties when invoking Eiffel features that are not expressed through the contracts. This is specially true when it comes to express what does not change through feature invocation. This makes the formal verification of Eiffel impossible, as in the case of verification we have to assume the worst possible implementation that still satisfies the contract.

To overcome these lacks, two approaches will be used to add full functional specifications to Eiffel: Models and Frames.

- Models are mathematical structures which describe the abstract state of an object. These mathematical structures are built out of sets, relations, functions and sequences. The currently available library called MML (cf. [Wid04], downloadable from [MML]) provides this functionality. This model library is based on a typed set theory of finite sets.
- Dynamic Frame Contracts (DFC) (cf. [SO]) enable us to talk about infinite sets of objects. They are contractual expressions describing the read effect and the write effect of a feature. With DFC we can solve the *frame problem* (cf. Section 2.1) and exclude unwanted side effects.

This semester thesis is an effort to redevelop a major part of the EiffelBase Library to support Dynamic Frame Contracts and Models. The goal is to

redesign and implement the linked list class in the EiffelBase Library and all supporting classes to allow full functional specifications and therefore formal verification. The redesign of the EiffelBase Library structure should lead to a cleaner and smoother Base Library.

1.2 Related Work

1.3 Overview

In Chapter 2 we present a short overview over the concept of Frames. In Chapter 3 a short introduction into the concept of Models is given. In Chapter 4 the redesigned and reimplemented Library is presented. In particular the new Library using DFCs and Models is introduced (Section 4.1) and differences to the original EiffelBase Library are emphasized (Section 4.1.1). Finally we present our results, conclusions and ideas for future work in Chapter 5.

Chapter 2

Frames

In this Chapter the frame problem is introduced and a short overview over the topic of Frames and in particular Dynamic Frames is given. For more information see also [Kas06, SO, Sch06].

2.1 Frame problem

The frame problem, the problem of how to formalize framing requirements, is a central issue in formal logic, first studied in the context of artificial intelligence in the 1960s. It was initially formulated as the problem of expressing a dynamical domain in logic without explicitly specifying which conditions are not affected by an action. Artificial intelligence tries to model the logical thinking of a human being. For the brain it is fairly simple to connect many things logically. But for a computer or a robot, an infinite number of things has to be specified. The frame problem is that specifying only which conditions are changed by an action do not allow, in logic, to conclude that all other conditions are not changed. To solve this problem one could write so called frame axioms which are logic formulas which explicitly describe the non-effects of each action. But since most actions do not affect most properties this solution is not suitable for large systems.

The frame problem has then been taken up by philosophers. In philosophy, it is about rationality in general. Whether it is possible to limit the scope of the reasoning required to derive the consequences on an action. In computer science the problem gained importance concerning program verification. Program verification tools are but logic tools which analyze a pro-

gram. Knowing what the program should do, for example from a contract, and then decide if the program really does things right.

A significant aspect of the behavior of any operation is what parts of the world it leaves unchanged. Usually, a specification for an operation is split into a functional requirement, which describes what changes the specified operation brings about and a framing requirement, which describes the frame of the operation so the part of the world which the operation is allowed to change. Anything outside the frame is left unchanged. In a modular setting we don't know all the variables of the program at the time we specify a computation. So there is no proper way to write we change x but we don't touch y and z. The only way is to say we modify x in a contract of a method and this statement is then translated at the client. So if the client or the implementer of an interface introduces two new variables y and z, the statement we modify x is translated to we modify x and don't touch y and z. But in the presence of pointers, this translation might be unsound. Independence of variables is important when guarantee absence of unwanted aliasing. But the combination of two important features of object oriented programming, namely encapsulation and the support for pointers, makes the frame problem really hard to solve because the combination introduces the possibility of aliasing.

2.2 Dynamic Frames

The frame problem in general is to specify which variables are affected by a computation and which are not. It is important for program verification but difficult to solve when using the common object oriented programming features like encapsulation and pointers. Existing solutions concerning the frame problem guarantee the absence of unwanted aliasing but for the price of more or less strong restrictions for the programmer. [Kas06] introduced a formal theory that supports specification variables and pointers without programming restrictions. The specification language is strong enough to express the desired property that at the present state, the values of two variables are independent.

For Kassios, a frame is nothing more than a specification variable whose value is a set of memory locations. So the new contribution of Kassios is that the frame itself is part of the state. The frame is like a function depending on the state. It is not a constant but it is dynamic which means the set of memory locations can change. For example suppose we have a frame which

frames a whole list of elements. If we insert a new element, it is automatically framed too and its memory address is added to the frame variable. The virtual address of the frame or the specification variable respectively can be itself part of the frame. Which in fact means that frames could frame themselves.

Kassios introduced three new definitions or framing specifications: The first specification (Equation 2.1) is preservation which is satisfied by every computation that does not touch region f.

$$\Xi f := \sigma' \triangleright f = \sigma \triangleright f \tag{2.1}$$

The second specification (Equation 2.2) is modification which is satisfied by every computation that only touches region f or at most allocates new memory. That implies all allocated locations other than f are not touched.

$$\Delta f := \Xi(\text{Used} \setminus f) \tag{2.2}$$

The third specification (Equation 2.3) is the swinging pivot requirement which does not allow the frame f to increase in any other way than allocation of new memory.

$$\Lambda f := f' \subseteq f \cup \text{Unused} \tag{2.3}$$

Equation 2.4 states that a variable v depends only on locations in f. Leaving these locations untouched preserves the value of v.

$$f \textbf{ frames } v := \forall \sigma' \cdot \Xi f \Rightarrow v' = v \tag{2.4}$$

Independence of two variables can be expressed as disjointness of dynamic frames. Disjointness of frames is an important property. It is desired to establish and preserve disjointness. To preserve disjointness, dynamic frames usually frame themselfes.

$$f \textbf{ frames } x \ \wedge \ g \textbf{ frames } y \ \wedge \ \mathit{disjoint}[f; g] \ \wedge \ \Delta f \Rightarrow y' = y \tag{2.5}$$

$$\mathit{disjoint}[f; g] \ \wedge \ \Delta f \ \wedge \ \Lambda f \Rightarrow \mathit{disjoint}[f'; g'] \tag{2.6}$$

Listing 2.1: put feature of the CC_COLLECTION class.

```
feature -- Element change
  put (v: G) is
      -- Ensure that structure includes 'v'.
    require
      can_add_element: can_put (v)
    use
      use_own_representation: representation
    modify
      modify_own_representation: representation
    deferred
    ensure
      model_updated: model |=| old model.extended (v)
      model_correspondends: model.contains (v)
      confined representation
    end
```

2.3 Dynamic Frames in Eiffel

Every feature in every class in the new base library uses new contract mechanisms. They are called *use* and *modify* and are used just like normal contracts. An example is given in Listing 2.1. If a feature is a query, only the *use* clause is necessary. If a feature is a command, both *use* and *modify* are needed. Additionally, a command should provide in its *ensure* clause the statement '**confined** *frame_name*'.

use clause The *use* clause describes which frames are used by this feature.

modify clause The *modify* clause describes which frames are possibly modified by calling this feature.

Chapter 3

Models

Reusable components should have precise specifications of their functionality. The creation of such precise specifications is a difficult but important task because precise specifications enable the client to understand the requirements for the usage of these components. An additional benefit for the component's creator is implementation hiding. Such a decoupling of different views on a component (author versus client) is needed for reuse and evolution of these components. [Sch03] gives an overview over the problem of creating precise specifications.

The creation of specifications in general is error-prone and often components are under-specified or over-specified. If they are under-specified, they leave too much space for misleading assumptions. If they are over-specified, they possibly violate information hiding, making reuse and evolution nearly impossible. As already described in Section 2.1, the Frame Problem is the best known specification problem because it is nearly impossible to clearly describe what has not changed during execution of a feature.

Models are mathematical constructs that describe the abstract behavior of a data type without dealing with the actual implementation. Examples are sequences, sets, bags or relations. The idea was to enable precise specifications through the usage of models. Even Hoare in 1972 (cf. [Hoa72]) related program verification to abstract data types and models.

3.1 Mathematical Model Library (MML)

The mathematical model library (MML) resulted from the Master Thesis of Tobias Widmer (cf. [Wid04]) and an improved version can be downloaded from [MML]. This library introduced a way to make specifications complete by using pure mathematical models that describe the abstract behavior of data types.

Models are represented by immutable objects. Immutable objects are objects that never change once they are created. Therefore the identity of such a model objects is defined through its state. That means that two objects that have the same state (therefore have the same values) are considered equal. Operations on immutable objects do not change the object itself, but produce new objects based on the current object and the arguments.

The MML is used in this semester thesis to create models in classes and to be able to write model contracts for features of the new base library, see Section 3.2.

3.2 Models in Eiffel

Every class in the new base library contains a special query that constructs the model from the current state of the instance. This query should be side-effect free and should only create the model object. This query is named *model*.

An example of such a *model* query is given in Listing 3.1. Here a bag is chosen as a model for a general container to contain its elements. The bag is created using a default implementation from the MML. Then all elements of the container are added to the bag object using an intermediate linear representation.

Such a *model* query is then used in contracts of features of this class. An example is given in Listing 3.2 in the **ensure** clause.

Model queries themselves can be built from several sub-models. An example is given in Listing 3.3. Here the *model* is built from a *model_sequence* which represents the sequence itself with all elements and a *model_cursor* which represents the current cursor position of the internal cursor. The final *model* is a pair of *model_sequence* and *model_cursor*. Two traversable objects are therefore equal if and only if they contain the same elements in the same

Listing 3.1: model feature of the CC_CONTAINER class.

```
feature -- Model
  model: MML_BAG [G] is
      -- Model of a general container
    use
      use_own_representation: representation
    local
      linear: CC_LINEAR [G]
    do
      create {MML_DEFAULT_BAG [G]} Result.make_empty
      linear := linear_representation

      from
        linear.start
      until
        linear.off
      loop
        Result := Result.extended (linear.item)
        linear.forth
      end
    ensure
      result_not_void: Result /= Void
    end
```

Listing 3.2: put feature of the CC_COLLECTION class.

```
feature -- Element change
  put (v: G) is
      -- Ensure that structure includes 'v'.
    require
      can_add_element: can_put (v)
    use
      use_own_representation: representation
    modify
      modify_own_representation: representation
    deferred
    ensure
      model_updated: model |=| old model.extended (v)
      model_correspondends: model.contains (v)
      confined representation
    end
```

order and the internal cursor is at the same location.

Listing 3.3: Model and sub-models from the CC_TRAVERSABLE class.

```
feature -- Model
  model_sequence: MML_SEQUENCE [G] is
      -- Model of a general traversable structure
    use
      use_own_representation: representation
    local
      linear: CC_LINEAR [G]
    do
      create {MML_DEFAULT_SEQUENCE [G]} Result.
         make_empty
      linear := linear_representation

      from
        linear.start
      until
        linear.off
      loop
        Result := Result.extended (linear.item)
        linear.forth
      end
    ensure
      result_not_void: Result /= Void
    end

  model_cursor: INTEGER is
      -- Model of a cursor for traversal
    use
      use_own_representation: representation
    deferred
    end

  model: MML_PAIR [MML_SEQUENCE [G], INTEGER] is
      -- Model of the traversable structure
    use
      use_own_representation: representation
    do
      create {MML_DEFAULT_PAIR [MML_SEQUENCE [G],
        INTEGER]} Result.make_from (model_sequence,
         model_cursor)
    ensure
      result_not_void: Result /= Void
    end
```

Chapter 4

New library with Complete contracts

One part of this semester thesis was the redesign of the EiffelBase Library (cf. [Eif05]). The goal was to get a cleaner and smoother base library. Of course the new base library should use Dynamic Frame Contracts and Models as discussed in Section 2.3 and Section 3.2. The redesign was focused on linked list class and all supporting classes.

The current EiffelBase Library is over engineered and sometimes messy. The usage of multi inheritance is somewhat a

4.1 EiffelBase with Complete Contracts

As stated above the new base library was focused on the linked list class and all supporting classes. In total there are about 20 classes in the library. Every class has a 'CC_' prefix in front of the name to be able to use this library in parallel to the original EiffelBase Library.

4.1.1 Differences to EiffelBase

Actually there are a lot of differences to the original EiffelBase Library. We try to give a short overview over these changes and explain the basic concepts and ideas that led to these differences.

Finite containers In contrast to the EiffelBase Library the new library is always finite. To be more precise, only finite containers are available.

TABLE not a heir of BAG The TABLE class is no longer a heir of the BAG class. Originally in the EiffelBase Library the idea was that a table is a bag of elements which can be directly accessed through an index. But for us the mathematical concept behind a bag and a table are too different to be directly related.

ACTIVE not a heir of BAG

INDEXABLE not a heir of TABLE

Feature renaming Some features in the original EiffelBase Library have names which do not reflect the actual meaning or which are not compliant with the Eiffel style guidelines (cf. Chapter 26 in [Mey97]).

- Feature cursor in CURSOR_STRUCTURE class has been renamed to cursor_position because it actually describes a position.
- Features isfirst and islast in the LIST class have been renamed to is_first and is_last to comply with the Eiffel style guidelines.
- TODO: replace_item_with statt nur replace etc noch anderes., remove_item statt nur remove, put_at_end statt nur put

BILINEAR class In the EiffelBase Library the BILINEAR class inherits two times from the LINEAR class. One time to actually inherit features which are then redefined to fit the bilinear case. The other time to be able to use features of the original linear class inside the implementation.

extend vs. put vs. force We tried to really define and implement differences between these features which are doing essentially the same but having different meanings.

- extend is defined as the non-defensive version of putting a new element into a container. This implies the strongest preconditions.
- put is defined as the medium-defensive version. This is more or less the mathematical point of view. For example if an element is already included in a set, the operation is just ignored.
- force is defined as the defensive version. This implies the weakest preconditions. This feature does everything to ensure a successful operation. The force feature would for example double an array if there is no more free space in it.

To be able to realize these different behaviors, new flags have been introduced. In particular can_extend(x) and can_put(x) are added to the preconditions of extend and put.

Fill feature discarded The fill feature has been discarded in the COLLECTION class because this would have introduced some nasty model contracts including under-specifications.

Command-query separation A strict command-query separation has been introduced. This was not the case in the original EiffelBase Library. Some features, mainly in the LINEAR class, were queries but in fact they move the internal cursor which obviously should not be the case. Queries are defined as side-effect free. The solution used in the new library was to save the internal cursor, do the operation and then move the cursor back to the old position. This was also necessary for the new Dynamic Frame Contracts (cf. Section 2.3) because a query has only a use clause, a command has a use and a modify clause.

CURSOR renamed to CURSOR_POSITION Because the cursor in the EiffelBase Library is in fact a cursor position, the corresponding class in the new library has been named CURSOR_POSITION. As stated above, the feature cursor has also been renamed to cursor_position.

Discarded classes Some classes which are supporting classes of the linked list class in the EiffelBase Library have been discarded in the new library.

- The classes BOX, FINITE and UNBOUNDED have been discarded because as stated above, only finite containers are available.
- The CHAIN class has been discarded and all features have been directly included in the LIST class.
- The DYNAMIC_CHAIN class has been discarded and all features have been directly included in the DYNAMIC_LIST class.

4.2 Linked List class

Chapter 5

Results and Conclusions

5.1 Usage of Frames and Models in Eiffel

Bibliography

[Eif05] Eiffel Software. *EiffelBase Library*, August 2005. http://archive.eiffel.com/products/base/.

[Hoa72] C.A.R. Hoare. Proof of correctness of data representations. *Acta Informatica*, 1(4):271–281, 1972.

[Kas06] Ioannis T. Kassios. Dynamic frames: Support for framing, dependencies and sharing without restrictions. *Formal Methods*, 2006.

[Mey92a] Bertrand Meyer. Applying 'design by contract'. *Computer*, 25(10):40–51, October 1992.

[Mey92b] Bertrand Meyer. *Eiffel: the language*. Object-Oriented Series. Prentice Hall, New York, NY, 1992.

[Mey97] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2 edition, 1997.

[MML] *MML*. http://se.inf.ethz.ch/people/schoeller/mml.html.

[Sch03] Bernd Schoeller. Strengthening eiffel contracts using models. In Hung Dang Van and Zhiming Liu, editors, *Proceeding of the Workshop on Formal Aspects of Component Software FACS'03*, number 284 in UNU/IIST Report, pages 143–158, September 2003.

[Sch06] Bernd Schoeller. Eiffel0: An object-oriented language with dynamic frame contracts. Technical Report 542, Chair of Software Engineering, Department of Computer Science, ETH Zurich, December 2006.

[SO] Bernd Schoeller and Jonathan Ostroff. Dynamic frame contracts. to be published.

[Wid04] Tobias Widmer. Reusable mathematical models. Master's thesis, ETH Zurich, July 2004.